AWS / DEVOPS PROJECT ASSIGNMENT

Microservices CI/CD Automation, AWS Deployment and Monitoring
Duration: 4 Working Days

Purpose

This assignment focuses exclusively on DevOps implementation. Participants will use a provided or existing three-microservice application and take responsibility for source control workflow, containerization, infrastructure provisioning, CI/CD automation, deployment, monitoring, security and troubleshooting. Application feature development is outside the scope of this assignment.

Project Scenario

A company has an existing application consisting of three microservices: User Service, Product Service and Order Service. The DevOps team must prepare the deployment environment and implement an automated, repeatable release process to deploy the services to AWS.

1. Git & Source Control Management

Expected tasks:

- Create or configure a GitHub repository for the supplied microservices application and DevOps configuration files.
- Review the existing repository structure and document the deployment components.
- Implement a suitable branch workflow for development and release changes.
- Configure GitHub webhooks or an equivalent trigger for Jenkins.
- Manage configuration changes through commits and pull requests.
- Ensure secrets, credentials and private keys are not committed to the repository.

2. Linux & AWS Environment Preparation

Expected tasks:

- Prepare an Ubuntu-based AWS EC2 instance for hosting the application containers.
- Configure SSH access, users, file permissions, package management and required Linux services.
- Install and configure Docker Engine, Docker Compose and AWS CLI.
- Configure EC2 security groups with only the required inbound and outbound access.
- Configure IAM permissions or an EC2 instance role for secure access to Amazon ECR.
- Review system processes, disk usage, CPU and memory utilization.

3. Docker & Container Orchestration

Expected tasks:

- Review the supplied application Dockerfiles and build images for all three microservices.
- Apply consistent image tagging and versioning standards.
- Configure Docker Compose to run the three services as a multi-container application.
- Configure container networks, environment variables, persistent volumes and restart policies.

- Configure container health checks and service dependencies.
- Configure Nginx as a reverse proxy to route requests to the appropriate microservice.
- Inspect container logs, diagnose failures and validate container-to-container connectivity.

4. Jenkins & CI/CD Pipeline Automation

Expected tasks:

- Install or configure Jenkins on an existing workstation or an available build server.
- Integrate Jenkins with the GitHub repository using a webhook or polling trigger.
- Create a Jenkinsfile to automate the release workflow.
- Configure pipeline stages for source checkout, configuration validation, image build and image tagging.
- Run available application tests as a pipeline validation step without modifying application features.
- Authenticate securely with Amazon ECR and push versioned images.
- Automate deployment to EC2 using SSH or another secure deployment method.
- Add pipeline failure handling, build logs and deployment verification.

5. Terraform & AWS Infrastructure Automation

Expected tasks:

- Write Terraform configuration to provision the required AWS resources.
- Provision one EC2 instance, three ECR repositories, security groups and required IAM resources.
- Use Terraform variables and outputs to make the configuration reusable.
- Execute terraform init, terraform validate, terraform plan and terraform apply.
- Review Terraform state handling and avoid storing sensitive values in source control.
- Document infrastructure changes and demonstrate safe resource cleanup using Terraform.

6. Deployment & Release Management

Expected tasks:

- Configure the EC2 host to authenticate to Amazon ECR using an IAM role where practical.
- Pull the approved image versions from ECR.
- Deploy or update all three microservices using Docker Compose.
- Verify Nginx routing and confirm that service health endpoints respond successfully.
- Demonstrate an application update through the Jenkins pipeline.
- Document the deployment procedure and a practical rollback or recovery approach.

7. Monitoring, Logging & Troubleshooting

Expected tasks:

- Install and configure Prometheus to collect host and available application metrics.
- Configure Node Exporter or an equivalent exporter for EC2 system metrics.
- Configure Grafana and connect it to Prometheus as a data source.
- Create dashboards for CPU, memory, disk usage, service availability and available application metrics.
- Review Docker container logs and relevant Linux system logs.
- Simulate a container failure, failed deployment or unavailable service.
- Identify the root cause, restore service and document the troubleshooting steps.

8. Major Evaluation Task

The primary evaluation will be based on an end-to-end DevOps implementation. Participants should be able to:

- Check out the existing microservices application from GitHub.
- Build and tag container images for all three services.
- Run the services using Docker Compose and validate service health.
- Provision the required AWS infrastructure using Terraform.
- Publish container images to Amazon ECR.
- Execute a Jenkins pipeline that deploys the selected image versions to AWS EC2.
- Verify application routing, service availability and deployment success.
- Use monitoring and logs to identify and resolve a simulated issue.

9. Expected Deliverables

- GitHub repository URL containing DevOps configuration and deployment files.
- Dockerfiles, Docker Compose configuration and Nginx configuration.
- Jenkinsfile and evidence of successful pipeline execution.
- Terraform files, plan output and infrastructure deployment evidence.
- Amazon ECR repositories with versioned images.
- Evidence of the three services running on AWS EC2.
- Prometheus configuration and Grafana dashboard screenshots.
- Deployment, rollback/recovery and troubleshooting notes.
- README documenting prerequisites, architecture, deployment steps and cleanup procedure.

10. Resource & Security Guidelines

- Use a minimal AWS footprint: one EC2 instance and three ECR repositories are sufficient for this assignment.
- Run Jenkins on an existing workstation or build machine where possible to avoid additional EC2 costs.
- Use Docker Compose; Kubernetes, EKS, RDS, NAT Gateway and load balancers are not required.
- Use IAM roles and Jenkins credentials instead of hardcoding AWS keys or passwords.
- Restrict SSH and administrative access to approved IP addresses; do not expose Jenkins or monitoring dashboards publicly.
- Check AWS pricing and account eligibility before provisioning. Configure a budget alert and remove unused resources after completion.

Additional Expectations

Participants are expected to demonstrate practical DevOps skills, independent troubleshooting, ownership of assigned work, secure configuration management, clear documentation and the ability to explain the complete CI/CD and deployment workflow.

Project Completion

The assignment must be completed within four working days. Assessment will focus on successful infrastructure automation, containerization, CI/CD implementation, AWS deployment, monitoring and troubleshooting. Participants are not expected to develop the microservices application itself; a supplied or existing application must be used.

Final outcome: A repeatable DevOps delivery process that deploys and monitors three existing microservices on AWS EC2 using GitHub, Jenkins, Docker, Terraform, Amazon ECR, Prometheus and Grafana.